

Jupyter + IPython Basics

This page is a student reading reference for the in-class demo on Jupyter and IPython. It summarizes the core ideas, vocabulary, and beginner commands you should know before doing notebook-based work in Python.

Main idea: Jupyter lets you keep *code*, *output*, *explanation*, and *visualization* together in one place. That is why notebooks are so useful for learning, exploration, and data science.

Contents

- What Jupyter is
- What IPython is
- Notebook files and the kernel
- Cells and execution
- Code cells vs Markdown cells
- Basic Markdown
- IPython magics
- Help and inspection
- Notebook shortcuts
- Common beginner mistakes

1) What is Jupyter?

Jupyter is an **interactive computing environment**. In one notebook, you can combine:

Code

Write Python and run it cell by cell.

Output

See printed results, tables, and errors immediately.

Explanation

Use text to explain what your code is doing.

Visuals

Display plots, images, and charts inline.

Jupyter supports the idea of **literate programming**: code and explanation should live together, not as separate disconnected files.

2) What is IPython?

IPython means **Interactive Python**. It is an enhanced Python environment that makes working interactively easier than using plain Python alone.

Tab completion

Helps finish variable names and methods.

Magic commands

Special shortcuts such as `%timeit`.

History

Lets you reuse earlier commands.

Better feedback

Improved tracebacks and debugging help.

In a Python notebook, **Jupyter uses IPython as the execution engine** for running your code.

3) Notebook files and the kernel

Notebook files

A Jupyter notebook file usually ends with `.ipynb`. It is **not just a Python script**. It is a structured JSON document that can store code cells, markdown cells, outputs, and metadata.

```
example_notebook.ipynb
```

The kernel

The **kernel** is the running process that actually executes your code. When you run a cell, the code is sent to the kernel.

The kernel remembers variables. If you define `x = 10` in one cell, later cells can use `x` until the kernel is restarted.

```
x = 10
```

```
x + 5
```

4) Cells and execution order

A notebook is made of **cells**. You usually work with code cells and markdown cells.

Cell Type	Purpose	What Happens
Code cell	Run Python code	Sent to the kernel and produces output or errors
Markdown cell	Write notes, headings, explanations	Rendered as formatted text in the notebook

Execution order matters

Notebooks are **not strictly top-to-bottom unless you run them that way**. You can run cell 8 before cell 3. That flexibility is useful, but it can also cause confusion.

A notebook may look correct on the page but fail when restarted and run from the top if cells were executed out of order.

5) Running cells

These are the most common actions students should know:

Action	Shortcut / Meaning
Run current cell	Shift + Enter
Run cell, stay in place	Ctrl + Enter
Run cell, insert cell below	Alt + Enter
Interrupt long-running code	Use the stop/interrupt button
Restart kernel	Clears all variables from memory

```
print("Hello, Jupyter!")
```

6) Code cells vs Markdown cells

Code cells

- Executed by the kernel
- Produce output
- Can change variable state

Markdown cells

- Rendered in the browser
- Not executed as Python
- Used for notes, structure, and math

```
# This is Python code
name = "Alice"
print(name)
```

```
# This is a Markdown idea, not Python
## Section Title
- point 1
- point 2
```

7) Markdown basics

Markdown is a simple plain-text syntax used to format text. It is very useful in notebooks because it helps you explain your work clearly.

You type	Meaning
# Title	Main heading
## Subtitle	Second-level heading
bold	Bold text

You type	Meaning
<code>*italic*</code>	Italic text
<code>- item</code>	Bullet list
<code>`code`</code>	Inline code

```
# My Analysis
## Goal
Understand the dataset.

**Important:** clean the missing values first.

- load data
- inspect columns
- summarize findings
```

8) IPython magics

Magic commands are special IPython commands. They begin with `%` for a line magic or `%%` for a cell magic. They are **not normal Python syntax**.

Magic	Use
<code>%timeit</code>	Measure how long an expression takes
<code>%pwd</code>	Show current working directory
<code>%ls</code>	List files in the directory
<code>%who</code>	Show defined variables
<code>%%time</code>	Time the whole cell

```
%timeit sum(range(100000))
```

```
%%time
numbers = [x**2 for x in range(1000000)]
len(numbers)
```

9) Help and object inspection

Jupyter and IPython make it easy to ask for help while you work.

Syntax	What it does
<code>len?</code>	Shows basic help for an object
<code>len??</code>	Shows more detail when available
<code>help(len)</code>	Uses Python's help system

```
sum?
list.append?
help(print)
```

Good students ask the notebook for help early instead of guessing syntax for too long.

10) Useful notebook shortcuts and modes

Jupyter notebooks usually have two modes:

Edit mode

You are typing inside a cell.

Command mode

You are controlling notebook actions such as adding or deleting cells.

Shortcut	Meaning
Esc	Enter command mode

Shortcut	Meaning
Enter	Enter edit mode
A	Add cell above
B	Add cell below
M	Convert selected cell to Markdown
Y	Convert selected cell to Code
D, D	Delete selected cell

11) Common beginner mistakes

1. Forgetting execution order

A variable may exist only because a previous cell was run earlier.

2. Confusing Markdown and Python

Markdown formatting is not valid Python code.

3. Not restarting and re-running

Always test whether your notebook works from top to bottom.

4. Ignoring the kernel state

Restarting the kernel clears variables, imports, and results stored in memory.

Best practice: before submission, **Restart Kernel and Run All**. That is the fastest way to catch hidden notebook problems.

12) Quick checklist for students

- I know what a notebook file (`.ipynb`) is.
- I understand the role of the kernel.
- I can tell the difference between a code cell and a markdown cell.

- I know how to run a cell with `Shift + Enter` .
- I know at least one help command such as `object?` .
- I know that `%timeit` is an IPython magic.
- I understand why cell execution order matters.

Reading reference prepared from the in-class Jupyter/IPython basics demo and organized for student review.